

List Operations

Learning Objectives

- Perform advanced list operations including slicing, sorting, and searching
- Use list methods to transform and analyze collections of data
- Implement algorithms that operate on lists efficiently

Engage (5-7 minutes)

A music playlist lets you shuffle songs, sort by artist, filter by genre, and create sub-playlists. Lists in programming offer similar operations. Mastering list operations means you can manipulate data in powerful ways—sorting scores, filtering results, extracting subsets, and transforming data. Today we expand our toolkit of list operations.

Explore (10-12 minutes)

Given the list `[34, 12, 78, 56, 23, 89, 45, 67]`, try these operations: sort ascending, sort descending, reverse the list, find the index of 56, count how many numbers are greater than 50, and create a slice from index 2 to 6. Now try `sort()` versus `sorted()`—what is the difference? Experiment with the `key` parameter to sort by the last digit of each number.

Explain (10-12 minutes)

List operations include slicing (extracting sublists using `[start:end:step]`), sorting (arranging elements using `sort()` or `sorted()`), searching (finding elements with `index()` and `count()`), and transforming (using `map()`, `filter()`, and list comprehensions). The `sort()` method modifies the list in place, while `sorted()` returns a new list. The `key` parameter allows custom sorting criteria. Slicing creates a shallow copy of the selected portion, and negative indices count from the end.

Elaborate (8-10 minutes)

Implement these operations without built-in functions: binary search on a sorted list, removing duplicates while maintaining order, flattening a nested list, rotating a list by `k` positions, and finding the `k`-th largest element. Then use list comprehensions to transform data: convert a list of temperatures from Celsius to Fahrenheit, generate a list of perfect squares up to 100, and filter words longer than 5 characters from a sentence. Discuss time complexity for each operation.

Evaluate (5-8 minutes)

Given the list [5, 3, 8, 1, 9, 2, 7, 4, 6], write the output after: sorting, slicing [2:7], reversing, index(9), and count() of elements greater than 4. Then write a list comprehension that creates a new list containing only the odd numbers squared.